

# YDSP Senior 组模拟考试

Copyright @ 云斗学院 @ 北斗学友教育科技有限公司

本次考试的答题时间为两个小时，请各位选手自行掌握时间。

提交请访问 [yundouxueyuan.com](http://yundouxueyuan.com) 参加对应的比赛，将最终答案以程序的形式提交至对应题目。

更进一步的提交细节，请参考对应题目里的说明。

最后，本次模拟考试为纯公益目的，不得用于任何未经授权的盈利活动。

\* 祝考试顺利 \*

# 2026 云斗学院软件能力认证第一轮 (YDSP - Senior) 提高级 C++ 语言试题

考生注意事项:

- 试题纸一共 16 页, 满分 100 分。作答后请记录答案, 并按要求提交至对应比赛处。
- 考试过程中不得使用任何电子设备或查阅任何书籍资料。

## 1 选择题 (每题 2 分, 共 30 分)

### 1.1 第 1 题

下列算法中, 用于求最小生成树的是 ▲。

- A. KMP 算法
- B. Prim 算法
- C. Floyd 算法
- D. Tarjan 算法

### 1.2 第 2 题

对数组 39, 7, 36, 27, 80 使用  $Base = 8$  的基数排序使得从小到大有序, 第一轮结束后的结果是 ▲。

- A. 80, 36, 7, 27, 39
- B. 7, 27, 36, 39, 80
- C. 80, 27, 36, 39, 7
- D. 80, 27, 36, 7, 39

### 1.3 第 3 题

中缀表达式  $5 << 4 + 3 * 2 << 1$  等价于后缀表达式 ▲。

- A. 5 4 3 2 \* + << 1 <<
- B. 5 4 << 3 2 \* 1 << +
- C. 5 4 3 2 \* + 1 << <<
- D. 前三个答案都不对

### 1.4 第 4 题

yummy 编译 C++ 源代码 `game.cpp` 获得了可执行文件 A。假设他希望可执行文件从文件 B 输入, 文件 C 输出, 并且运行时在 `game.cpp` 的 `main` 函数内填入一个参数 "D", 那么他在 Linux 终端里输入的命令应该是 ▲。

- A. `./A D < B > C`
- B. `./A D < C > B`
- C. `./A D B C`
- D. `./A < D > B C`

### 1.5 第 5 题

考虑一个长为 2026 的字符串  $s$ ，使用  $s[l, \dots, r]$  表示  $s$  中第  $l$  到  $r$  个字符构成的子串， $r_i$  为最大的  $r \geq 0$  使得  $s[i-r, \dots, i+r]$  是回文串。如果  $r_{100} = 70$ ， $r_{80} = 60$ ，那么  $r_{120}$  至少是 ▲。

- A. 20
- B. 50
- C. 60
- D. 前三个选项都不对

### 1.6 第 6 题

在一片足够空旷的平地上，一个微型机器人以“前进 1 厘米，左转  $60^\circ$ ，前进 1 厘米，左转  $60^\circ$ ，前进 1 厘米，右转  $60^\circ$ ，前进 1 厘米，右转  $30^\circ$ ”为周期运动。那么，它第一次回到起点时，运动的总路程是 ▲ 厘米。

- A. 48
- B. 12
- C. 4
- D. 前三个答案都不对

### 1.7 第 7 题

现有两个 `int` 型变量  $a, b$ ，满足  $0 < b < a \leq 10^9$ 。令  $p$  为  $a, b$  按位异或的结果， $q$  为  $a, b$  的最大公因数，那么 ▲。

- A.  $p \geq q$ ，等号可能取到。
- B.  $p \leq q$ ，等号可能取到。
- C.  $p > q$ 。
- D.  $p < q$ 。

### 1.8 第 8 题

对于集合  $A, B$ ，定义  $A \oplus B = \mathbb{C}_{A \cup B} A \cap B$ 。

给出函数  $f(S) = \begin{cases} 0 & S = \emptyset \\ \min_{x=1}^n f(S \oplus \{x\}) + w(S, x) & S \neq \emptyset \end{cases}$ ，其中  $w(S, x) \geq 0$ 。在不假设  $w$  的任何其他性质时，要求求解  $f(\{1, \dots, n\})$ ，下列算法中最合适的是 ▲。

- A. 动态规划
- B. 贪心法
- C. Dijkstra 算法
- D. Floyd 算法

### 1.9 第 9 题

一张  $n$  ( $n \geq 4$ ) 个结点  $m$  条边的简单无向连通二分图具有欧拉回路。那么下列说法**错误**的是 ▲。

- A. 若结点 1 在左部，则有且仅有一种方法把所有结点划分为左部和右部。
- B. 左部和右部均至少有 2 个结点。

- C.  $m$  一定是偶数。
- D.  $n$  一定是偶数。

阅读以下材料，完成第 10 至 12 题。

维护颜色段是一个经典的问题。对于一个序列  $a_1, \dots, a_n$ ，如果  $a_l, a_{l+1}, \dots, a_r$  全部相等，并且  $a_{l-1} \neq a_l, a_r \neq a_{r+1}$ （特别地，认为  $a_0 = a_{n+1} = +\infty$ ），则称  $[l, r]$  为一个颜色段。例如，1, 1, 5, 3, 3, 1 共有四个连续段  $[1, 2], [3, 3], [4, 5], [6, 6]$ 。

现在我们需要一种数据结构，给出序列  $a_1, \dots, a_n$ ，需要支持区间赋值、区间数颜色段个数，且总操作次数为  $O(n)$  次。Alice 和 Bob 选用了不同的方法。

### 1.10 第 10 题

Alice 选择使用线段树维护一个序列  $b_i = \begin{cases} 0 & a_i = a_{i+1} \\ 1 & a_i \neq a_{i+1} \end{cases}$ ，支持单点修改、区间求和。下列说法

正确的是     ▲    。

- A. 她写的线段树必须有懒标记 (lazy-tag)。
- B. 这棵线段树如果使用树状数组代替，那么时间复杂度增加。
- C. 如果要求子串  $a[100, \dots, 200]$  的颜色段个数，那么需要计算序列  $b$  在  $[100, 199]$  上的和。
- D. 使用这棵线段树，还能对于每个下标  $i$  查询它所在颜色块的右端点，但是时间复杂度为  $\Theta(\log^2 n)$ 。

### 1.11 第 11 题

Bob 选择维护一个 `set<int> s` 记录所有颜色段的左端点，然后每次查询时遍历区间内的颜色块实现计数，称为“珂朵莉树”。想要查询包含下标  $i$  所在的颜色块右端点  $r$ ，表达式正确的是     ▲    。

- A. `r=--s.lower_bound(i)`
- B. `r=*s.lower_bound(i)-1`
- C. `r=--s.upper_bound(i)`
- D. `r=*s.upper_bound(i)-1`

### 1.12 第 12 题

Alice 和 Bob 在讨论各自做法的时间复杂度。下列说法正确的是     ▲    。

- A. 使用 Alice 的做法，每次操作最坏  $O(\log n)$ 。
- B. 使用 Alice 的做法，总时间复杂度为  $O(n \log n)$ 。
- C. 使用 Bob 的做法，每次操作最好  $O(n)$ 。
- D. 使用 Bob 的做法，如果输入的区间、操作类型、赋值均随机，总时间复杂度为  $O(n^2)$ 。

### 1.13 第 13 题

已知  $p$  为一个  $1 \sim 6$  的全排列，对任意  $1 \leq i \leq 6$  均有  $p(p(p(p(i)))) = i$ 。这样的排列  $p$  有     ▲     种。

- A. 256
- B. 301
- C. 376

D. 456

### 1.14 第 14 题

同余方程  $x^3 \equiv 909 \pmod{1001}$  有 ▲ 组解满足  $0 \leq x < 1001$ 。

A. 1

B. 3

C. 9

D. 27

### 1.15 第 15 题

一棵二叉树的前序遍历为  $C, 1, D, 8, 4, 5, 3, A, 2, 7, 6, B$ , 中序遍历为  $4, 8, D, 3, 5, 2, A, 7, 1, C, 6, B$ , 其中  $A, B, C, D$  中  $9 \sim 12$  各出现了一次。

定义  $LCA(x, y)$  为  $x, y$  的最近公共祖先。已知  $LCA(4, 9) = 12$ ,  $LCA(2, 10) = C$ ,  $LCA(1, 11) = 11$ , 那么  $A, B, C, D$  中, 最大的数字是 ▲。

A.  $A$

B.  $B$

C.  $C$

D.  $D$

## 2 阅读程序 (无特殊说明时判断 1.5 分, 选择 3 分, 共 40 分)

### 2.1 第 1 题 (12 分)

```
1 #include <stdio>
2 using namespace std;
3
4 const int N=1.01e6,mod=998244353;
5 char s[N];
6 int n,m,p[26][26];
7
8 signed main(){
9     scanf("%d%d",&n,&m);
10    scanf("%s",s+1);
11    for(int i=1;i<n;i++){
12        ++p[s[i]-'a'][s[i+1]-'a'];
13    }
14    for(int i=1,x=s[n]-'a';i<=m;i++){
15        int mx=0,y=0;
16        for(int j=0;j<26;j++){
17            if(p[x][j]>mx){
18                mx=p[x][j];
19                y=j;
```

```
20     }  
21     }  
22     printf("%c",y+'a');  
23     x=y;  
24 }  
25 return 0;  
26 }
```

输入数据保证： $1 \leq n, m \leq 10^6$ ， $s$  是长度为  $n$  的小写英文字母字符串。

### 2.1.1 判断题

1. 程序输出的字符串长度取决于  $n$  的大小。
2. 若将代码第 15 行 `mx=0` 更换为 `mx=-1`，程序的输出结果不变。
3. (2 分) 若将代码第 17 行的 `>` 更换为 `>=`，程序的输出结果不变。

### 2.1.2 选择题

4. 程序输出的第一个字符取决于 \_\_\_\_▲\_\_\_\_。  
A.  $s$  的第一个字符。  
B.  $s$  的最后一个字符。  
C.  $s$  的最后一个字符在  $s$  中所有出现位置的下一个位置出现次数最多的字符，出现次数相同则选字典序较小者。  
D.  $s$  的最后一个字符在  $s$  中所有出现位置的下一个位置出现次数最多的字符，出现次数相同则选字典序较大者。
5. (4 分) 若输入为 10 7 daedacbage 则输出为 \_\_\_\_▲\_\_\_\_。  
A. dacbacb  
B. daedaed  
C. acbacba  
D. aebaeba

## 2.2 第 2 题 (14 分)

```
1 #include <stdio>  
2 using namespace std;  
3  
4 const int N=1.01e6;  
5 int n,m,ans;  
6 int fa[N],w[N];  
7 int stk[N],top;
```

```
8
9 void Find(int x){
10     if(fa[x]!=x){
11         Find(fa[x]);
12         w[x]^=w[fa[x]];
13         fa[x]=fa[fa[x]];
14     }
15     return;
16 }
17
18 void init(){
19     for(int i=1;i<=n;i++){
20         fa[i]=i;
21         w[i]=0;
22     }
23     return;
24 }
25
26 signed main(){
27     scanf("%d%d",&n,&m);
28     init();
29     ans=1;
30     for(int i=1,u,v,x;i<=m;i++){
31         scanf("%d%d%d",&u,&v,&x);
32         Find(u),Find(v);
33         if(fa[u]==fa[v]&&(w[u]^w[v])!=x){
34             do{
35                 fa[stk[top]]=stk[top];
36                 w[stk[top]]=0;
37             }while(--top);
38             ++ans;
39         }
40         if(fa[u]!=fa[v]){
41             int _w=(w[u]^w[v]^x);
42             u=fa[u],v=fa[v];
43             fa[v]=u;
44             w[v]=_w;
45             stk[++top]=v;
46         }
47     }
48     printf("%d\n",ans);
49     return 0;
```

50 }

输入数据保证:  $1 \leq n, m \leq 10^6, 1 \leq u, v \leq n, u \neq v, x \in \{0, 1\}$ 。

### 2.2.1 判断题

1. 若将代码 34 ~ 37 行的整个 do-while 语句替换为 `init();`, 代码的时间复杂度不变。
2. 对于任何合法的输入数据, 该程序输出结果不会超过  $m$ , 且可能为  $m$ 。
3. (2 分) 代码 34 ~ 37 行的 do-while 语句, 可能导致变量 `top` 变为负数进而导致程序运行时错误。

### 2.2.2 选择题

4. (2 分) 假设将 **Find** 操作的均摊时间复杂度视为  $O(1)$ , 并且  $n, m$  同阶, 则该算法的时间复杂度可以视为 ▲。

- A.  $O(n)$
- B.  $O(n \log n)$
- C.  $O(n^2)$
- D.  $O(n\sqrt{n})$

5. 若输入为 4 5 1 2 1 2 3 1 3 4 1 4 1 0 4 1 1, 则输出为 ▲。

- A. 1
- B. 2
- C. 3
- D. 4

6. (4 分) 当  $n = 3, m = 4$  时, 有 ▲ 种合法的输入可以使该程序输出 1。

- A. 4992
- B. 4896
- C. 5184
- D. 5088

## 2.3 第 3 题 (14 分)

```
1 #include <stdio>
2 using namespace std;
3
4 const int N=128,mod=998244353;
5 int n,m;
```

```
6 char s[N],t[N];
7 int bd[N],to[N][26];
8 int f[N][N][N][2],sum[N][N][N][2];
9 int ans;
10
11 signed main(){
12     scanf("%d%d",&n,&m);
13     scanf("%s",s+1);
14     scanf("%s",t+1);
15
16     for(int i=2,j=0;i<=n;i++){
17         while(j>0&&s[i]!=s[j+1])j=bd[j];
18         if(s[i]==s[j+1])++j;
19         bd[i]=j;
20     }
21
22     for(int i=0;i<=n;i++){
23         for(int c=0;c<26;c++){
24             int x=i;
25             while(x>0&&s[x+1]-'a'!=c)x=bd[x];
26             if(s[x+1]-'a'==c)++x;
27             to[i][c]=x;
28         }
29     }
30
31     f[0][0][0][0]=1;
32     for(int x=0;x<=m;x++){
33         for(int y=0;y<=m;y++){
34             for(int k=0;k<=n;k++){
35                 for(int tag=0;tag<2;tag++){
36                     if(x>0&&y>0&&t[x]==t[y]){
37                         int _k=to[k][t[x]-'a'];
38                         int _tag=(tag|(_k==n?1:0));
39                         (f[x][y][_k][_tag]+=sum[x-1][y-1][k][tag])%=mod;
40                     }
41                 }
42             }
43             for(int k=0;k<=n;k++){
44                 for(int tag=0;tag<2;tag++){
45                     int *p=&sum[x][y][k][tag];
46                     (*p)=f[x][y][k][tag];
47                     if(x>0)
```

```

48         ((*p)+=sum[x-1][y][k][tag])%=mod;
49         if(y>0)
50             ((*p)+=sum[x][y-1][k][tag])%=mod;
51         if(x>0&&y>0)
52             ((*p)-=sum[x-1][y-1][k][tag])%=mod;
53     }
54 }
55 }
56 }
57
58 for(int k=0;k<=n;k++){
59     (ans+=sum[m][m][k][1])%=mod;
60 }
61 if(ans<0)ans+=mod;
62 printf("%d\n",ans);
63 return 0;
64 }

```

输入数据保证：  $1 \leq n, m \leq 100$ ， $s, t$  分别是长度为  $n, m$  的小写英文字母字符串。

### 2.3.1 判断题

1. 当字符串  $s$  为 `aabaaab` 时 `bd` 序列为 `{0,1,0,1,2,2,3}`。
2. 代码 61 行的判断是没有必要的，在执行 61 行前变量 `ans` 不可能为负数。
3. (2 分) 若  $s$  不是  $t$  的子序列，输出结果一定为 0。

### 2.3.2 选择题

4. (2 分) 若代码 25 行 `while` 循环中删除 `x>0` 的判断，程序可能出现的错误是     ▲    。
  - A. 输出答案错误 (WA)
  - B. 超出时间限制 (TLE)
  - C. 超出空间限制 (MLE)
  - D. 运行时错误 (RE)
5. 若输入为 `3 8 csp aaacspaa`，则输出为     ▲    。
  - A. 102
  - B. 144
  - C. 120
  - D. 160
6. (4 分) 当  $m = 5$  时，程序输出值最大可能是     ▲    。

- A. 249
- B. 251
- C. 253
- D. 255

### 3 完善程序（单选题，每小题 3 分，共计 30 分）

#### 3.1 组合问题（15 分）

构造一个长度为  $m$  的序列  $a_1, a_2, \dots, a_m$ ，每个元素都是不超过  $n$  的正整数，其中整数  $i$  恰好有  $x_i$  个， $m = \sum_{1 \leq i \leq n} x_i$ 。

求出满足要求的不同序列数量。

以下代码求解了上述问题。试补全程序。

```
1 #include <stdio>
2 #define int long long
3 using namespace std;
4
5 const int N=1.01e6,mod=998244353;
6 int fac[N],ifac[N];
7
8 int fpow(int x,int k=/* Blank 1 */){
9     int res=1;
10    while(k){
11        if(k&1ll)res=res*x%mod;
12        x=x*x%mod;
13        k/=2;
14    }
15    return res;
16 }
17
18 int inv(int x){
19     return fpow(x);
20 }
21
22 void init(){
23     fac[0]=1;
24     for(int i=1;i<N;i++)fac[i]=fac[i-1]*i%mod;
25     ifac[N-1]=inv(fac[N-1]);
26     for(int i=N-2;i>=0;i--)/* Blank 2 */;
27     return;
28 }
```

```
29
30 int C(int x,int y){
31     if(x<y)return 0;
32     return /* Blank 3 */;
33 }
34
35 int n,x[N],sum;
36
37 signed main(){
38     init();
39     scanf("%lld",&n);
40     int ans=/* Blank 4 */;
41     for(int i=1;i<=n;i++){
42         scanf("%lld",&x[i]);
43         sum+=x[i];
44         /* Blank 5 */;
45     }
46     printf("%lld\n",ans);
47     return 0;
48 }
```

1. /\* Blank 1 \*/ 处应该填 \_\_\_\_▲\_\_\_\_。

- A. 0
- B. mod
- C. mod-1
- D. mod-2

2. /\* Blank 2 \*/ 处应该填 \_\_\_\_▲\_\_\_\_。

- A. ifac[i]=ifac[i+1]\*(i)%mod
- B. ifac[i]=ifac[i+1]\*(i+1)%mod
- C. ifac[i]=ifac[i+1]\*inv(i)%mod
- D. ifac[i]=ifac[i+1]\*inv(i+1)%mod

3. /\* Blank 3 \*/ 处应该填 \_\_\_\_▲\_\_\_\_。

- A. fac[x]\*ifac[y]%mod
- B. fac[x]\*ifac[x-y]%mod
- C. fac[x]\*ifac[y]%mod\*ifac[x-y]%mod
- D. fac[x-y]\*ifac[y]%mod

4. /\* Blank 4 \*/ 处应该填 \_\_\_\_▲\_\_\_\_。

- A. 0

- B. 1
- C. fac[n]
- D. ifac[n]

5. /\* Blank 5 \*/ 处应该填 \_\_\_\_▲\_\_\_\_。

- A. (ans\*=C(sum,x[i]))%=mod
- B. (ans+=C(sum,x[i]))%=mod
- C. (ans\*=C(sum+x[i],x[i]))%=mod
- D. (ans+=C(sum+x[i],x[i]))%=mod

### 3.2 边权差最短路 (15 分)

给定一个含  $n$  个点  $m$  条边的带权无向图，边权为整数，起点为 1，终点为  $n$ ，保证至少存在一条从 1 到  $n$  的路径。

对于一条从起点到终点的路径，定义该路径的花费为：将经过的所有边权按顺序写下来后，该序列相邻元素差的绝对值之和。

求从 1 到  $n$  最小总费用。

以下代码求解了上述问题，试补全程序。

```
1 #include <cstdio>
2 #include <queue>
3 #include <vector>
4 #include <algorithm>
5 #define int long long
6 using namespace std;
7
8 const int N=1.01e6,inf=1e18;
9
10 struct edge{
11     int to,len;
12     edge(int to=0,int len=0):to(to),len(len){}
13 };
14
15 struct node{
16     int pos,dis;
17     node(int pos=0,int dis=0):pos(pos),dis(dis){}
18     bool operator < (const node& _)const{
19         return /* Blank 1 */;
20     }
21 };
22
23 int n,m;
24 vector<node> e[N];
```

```
25 vector<edge> G[N];
26 int dis[N],vis[N];
27 priority_queue<node> Q;
28
29 void add_edge(int u,int v,int w){
30     G[u].push_back(edge(v,w));
31     G[v].push_back(edge(u,w));
32     return;
33 }
34
35 signed main(){
36     scanf("%lld%lld",&n,&m);
37     for(int i=1,u,v,w;i<=m;i++){
38         scanf("%lld%lld%lld",&u,&v,&w);
39         e[u].push_back(node(i,w));
40         e[v].push_back(node(i,w));
41     }
42     for(int i=1;i<=n;i++){
43         sort(e[i].begin(),e[i].end());
44         for(unsigned j=0;j<e[i].size();j++){
45             if(j+1<e[i].size())
46                 add_edge(e[i][j].pos,e[i][j+1].pos,/* Blank 2 */);
47             if(i==1)
48                 add_edge(m+1,e[i][j].pos,0);
49             if(i==n)
50                 add_edge(m+2,e[i][j].pos,0);
51         }
52     }
53     for(int i=1;i<=m+2;i++)dis[i]=inf;
54     dis[m+1]=0;
55     Q.push(node(/* Blank 3 */,0));
56     while(!Q.empty()){
57         int u=Q.top().pos;
58         Q.pop();
59         if(/* Blank 4 */continue;
60         vis[u]=1;
61         for(unsigned i=0;i<G[u].size();i++){
62             int v=G[u][i].to,w=G[u][i].len;
63             if(dis[u]+w<dis[v]){
64                 dis[v]=dis[u]+w;
65                 Q.push(node(v,dis[v]));
66             }
67         }
68     }
69 }
```

```
67     }  
68 }  
69 printf("%lld\n", /* Blank 5 */);  
70 return 0;  
71 }
```

1. /\* Blank 1 \*/ 处应该填 \_\_\_\_▲\_\_\_\_。

- A. pos<\_.pos
- B. pos>\_.pos
- C. dis<\_.dis
- D. dis>\_.dis

2. /\* Blank 2 \*/ 处应该填 \_\_\_\_▲\_\_\_\_。

- A. e[i][j].dis-e[i][j+1].dis
- B. e[i][j+1].dis-e[i][j].dis
- C. e[i][j].dis
- D. e[i][j].dis+e[i][j+1].dis

3. /\* Blank 3 \*/ 处应该填 \_\_\_\_▲\_\_\_\_。

- A. 1
- B. m
- C. m+1
- D. m+2

4. /\* Blank 4 \*/ 处应该填 \_\_\_\_▲\_\_\_\_。

- A. vis[u]==1
- B. dis[u]==inf
- C. u>n
- D. u>m

5. /\* Blank 5 \*/ 处应该填 \_\_\_\_▲\_\_\_\_。

- A. dis[n]
- B. dis[m]
- C. dis[m+1]
- D. dis[m+2]